

LAPORAN PRAKTIKUM 6

DOUBLE LINKED LIST

STRUKTUR DATA



Disusun Oleh:

Nama: Amiratul Fadhilah

NIM: 2511532023

Dosen Pengampu: Dr. Ir. Wahyudi, S. T., M. T

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah Yang Maha Esa atas rahmat dan karunia-Nya, sehingga penulisan laporan praktikum dengan judul “Double Linked List” ini dapat diselesaikan tepat waktu. Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas mata kuliah Praktikum Struktur Data.

Laporan ini akan membahas secara mendalam mengenai konsep, logika dasar, serta implementasi struktur data *Double Linked List* menggunakan bahasa pemrograman Java. Melalui pembahasan ini, diharapkan pembaca dapat memahami bagaimana pembuatan struktur node dasar, operasi penyisipan data, operasi penghapusan, hingga proses penelusuran data dari dua arah. Kemampuan navigasi dua arah inilah yang menjadi keunggulan utama dari DLL dibandingkan struktur data linear pendahulunya.

Penulis menyadari bahwa masih terdapat berbagai kekurangan, baik dari segi penulisan maupun analisis kode program. Oleh karena itu, kritik dan saran yang membangun dari dosen pengampu maupun asisten praktikum sangat diharapkan demi perbaikan di masa mendatang. Semoga dokumentasi praktikum ini dapat memberikan manfaat tambahan bagi pembaca.

Padang, 12 Mei 2026

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN	
2.1 Landasan Teori	3
2.2 Alat dan Bahan.....	3
2.3 Langkah Praktikum dan Kode Program	3
2.3.1 Kelas NodeDLL	3
2.3.2 Kelas InsertDLL	5
2.3.3 Kelas HapusDLL.....	8
2.3.4 Kelas PenelusuranDLL	10
BAB III PENUTUP	
3.1 Kesimpulan.....	13
3.2 Saran.....	13
DAFTAR PUSTAKA.....	14
TUGAS 6 PRAKTIKUM STRUKTUR DATA.....	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam ilmu komputer, pemahaman mengenai struktur data memegang peranan yang sangat esensial. Struktur data tidak hanya sekadar wadah penyimpanan, tetapi secara langsung menentukan efisiensi dan performa algoritma dalam memproses informasi di dalam memori komputer. Pemilihan fondasi penyimpanan yang tepat adalah kunci utama bagi pengembangan aplikasi yang optimal. Pada pertemuan praktikum sebelumnya, kita telah mempelajari struktur data *Single Linked List* (SLL), yang merupakan solusi cerdas untuk mengatasi sifat statis dan alokasi ruang kaku pada struktur data Array konvensional.

Meskipun SLL berhasil memberikan pengalokasian memori secara dinamis, metode ini membawa kelemahan fungsional, yaitu pergerakan pencarian data hanya dapat dilakukan ke satu arah saja (dari elemen depan menuju ke elemen belakang). Apabila sebuah program secara tiba-tiba membutuhkan penelusuran balik atau modifikasi elemen di posisi sebelumnya, program tersebut terpaksa harus memulai proses pencarian ulang dari elemen terdepan. Untuk mendobrak kelemahan mendasar tersebut, dikembangkanlah struktur data yang lebih mutakhir, yakni *Double Linked List* (DLL). Sesuai dengan namanya, DLL melipatgandakan fleksibilitas navigasi dengan menambahkan satu pointer ekstra ke arah berlawanan, memungkinkan elemen data untuk saling bertukar informasi dan ditelusuri ke dua arah dengan sama mulusnya.

1.2 Tujuan Praktikum

Tujuan utama dari pelaksanaan praktikum mengenai Double Linked List ini adalah sebagai berikut:

- 1) Memahami konsep teoritis maupun representasi visual dari struktur data *Double Linked List* (DLL) beserta arsitektur node yang menjadi penyusun utamanya.

- 2) Melatih keterampilan teknis pemrograman dalam merakit dan mengompilasi kode program Java guna mengendalikan manipulasi dua belah pointer (*next* dan *prev*).
- 3) Melatih kepekaan analisis logika tinggi saat mengeksekusi operasi vital seperti penyisipan (*insertion*) dan penghapusan data (*deletion*) agar tidak terjadi kondisi rantai putus yang menyebabkan kebocoran memori (*memory leak*).

1.3 Manfaat Praktikum

Melalui pelaksanaan praktikum ini, mahasiswa diharapkan dapat memetik ragam manfaat akademis dan teknis. Secara akademis, mahasiswa bisa melihat secara konkret implementasi nyata dari rumitnya teori manajemen alokasi memori dinamis dan interaksi referensi objek (*pointer*) yang selama ini diperdengarkan di ruang kelas perkuliahan. Teori yang abstrak kini terealisasi dalam barisan instruksi Java yang rasional.

Selain itu, diharapkan kapasitas pemecahan masalah (*problem-solving*) dan ketelitian sintaksis mahasiswa akan semakin terasah tajam. Mengingat struktur kode program DLL sangat bergantung pada kehati-hatian tingkat tinggi saat memindahkan urutan pengait antar objek, mahasiswa dilatih secara tidak langsung untuk berpikir komputasional, runut, dan sangat presisi. Keterampilan penguasaan memori dinamis ini merupakan pijakan yang sangat penting sebelum mahasiswa dihadapkan pada tugas pembuatan perangkat lunak berskala masif di waktu yang akan datang.

BAB II

PEMBAHASAN

2.1 Landasan Teori

Double Linked List (DLL) merupakan perluasan wujud dari struktur data dinamis linier di mana setiap entitas datanya (node atau simpul) diformat untuk menyimpan tiga fragmen utama: satu kompartemen berfungsi vital untuk menyimpan nilai data itu sendiri, sementara dua area lainnya berperan sebagai pointer yang mendapat tugas untuk merekatkan node tersebut ke node sekutu di depannya maupun di belakangnya. Arsitektur penunjuk ganda ini menciptakan sebuah rantai ikatan data dua arah yang sangat padu, meruntuhkan keterbatasan navigasi konvensional.

Eksistensi pointer yang difokuskan untuk menunjuk ke arah belakang (prev) pada DLL menyuguhkan akselerasi performa yang dominan dalam perkara fleksibilitas. Proses modifikasi data, intervensi penyisipan node baru, hingga pemusnahan node yang berada tepat di tengah- tengah rentetan senarai menjadi lebih irit langkah komputasi ketimbang SLL, lantaran program tidak lagi diwajibkan untuk memutar balik penelusuran dari pangkal (Head) hanya untuk menjangkau elemen sebelumnya [3]. Walau begitu, di balik keagungan navigasi dua dimensi ini, setiap node DLL mensyaratkan dedikasi kapasitas ruang memori yang sedikit lebih besar. Terlebih, algoritma pemrograman DLL menuntut disiplin yang kaku dari sisi programmer, karena kesalahan satu alur pointer saja berisiko mengkandaskan kelangsungan program secara fatal

2.2 Alat dan Bahan

- 1) Perangkat Keras (*Hardware*): Satu buah perangkat komputasi seperti PC atau Laptop.
- 2) Perangkat Lunak (*Software*): Instalasi OS berkecepatan stabil, ditambah presensi mesin virtual JDK dan IDE seperti Eclipse untuk kompilasi Java.

2.3 Langkah Praktikum dan Kode Program

2.3.1 Kelas NodeDLL

Tahap awal adalah mencetak desain dasar untuk satu node tunggal yang memiliki dua referensi. Ibarat gerbong kereta api, kelas ini

menentukan muatan apa yang akan dibawa dan seberapa banyak rantai pengait yang dimilikinya.

```
package pekan6_2511532023;

public class NodeDLL_2511532023 {
    // Mendefinisikan kelas node
    int data_2023; // Data
    NodeDLL_2511532023 next_2023; // Pointer ke next node
    NodeDLL_2511532023 prev_2023; // Pointer ke previous node

    // Konstruktor
    public NodeDLL_2511532023(int data_2023) {
        this.data_2023 = data_2023;
        this.next_2023 = null;
        this.prev_2023 = null;
    }
}
```

Kode Program 2.3.1

Penjelasan Kode:

- `int data_2023;`: Baris deklarasi variabel penampung. Bagian inilah yang bertugas memboyong atau menyimpan nilai bilangan riil yang dimasukkan oleh pengguna ke dalam "gerbong".
- `NodeDLL_2511532023 next_2023;`: Deklarasi tali pengait depan. Komponen ini berupa pointer yang berfungsi memegang tangan atau menunjuk lokasi presisi node kawan yang berdiri di depannya.
- `NodeDLL_2511532023 prev_2023;`: Deklarasi tali pengait belakang. Ini adalah inovasi mahkota dari *Double Linked List*, pointer ini berfungsi untuk menengok dan memegang tangan node rekanan yang berada di belakangnya.
- `Public NodeDLL_2511532023(int data_2023) {...}`: Ini adalah blok Konstruktor. Analogi sederhananya, ketika kita merakit satu gerbong node baru, kita wajib langsung menaruh muatan datanya melalui pintu masuk ini.
- `this.next_2023 = null;` dan `this.prev_2023 = null;`: Merupakan nilai bawaan (default). Mengingat sebuah node yang baru saja lahir masih terisolasi dan belum dikaitkan ke dalam antrean panjang, maka kedua pengait bajanya untuk sementara waktu diarahkan ke keheningan alias kosong (null).

2.3.2 Kelas InsertDLL

Tahap selanjutnya difokuskan pada perakitan node-node yang tercecer tadi menjadi satu tali rantai yang panjang. Modul ini memperlihatkan metode penyisipan dari berbagai gerbang: pintu masuk awal, pintu keluar paling ujung, hingga menyelip secara diam-diam di antara kerumunan antrean tengah.

```
package pekan6_2511532023;

public class InsertDLL_2511532023 {
    // Menambahkan node di awal DLL
    static NodeDLL_2511532023 insertBegin_2023(NodeDLL_2511532023
head_2023, int data_2023) {
        // Buat node baru
        NodeDLL_2511532023 new_node_2023 = new
NodeDLL_2511532023(data_2023);
        // Jadikan pointer nextnya head
        new_node_2023.next_2023 = head_2023;
        // Jadikan pointer prev head ke new node
        if (head_2023 != null) {
            head_2023.prev_2023 = new_node_2023;
        }
        return new_node_2023;
    }
    // Fungsi menambahkan node di akhir
    public static NodeDLL_2511532023
insertEnd_2023(NodeDLL_2511532023 head_2023, int newData_2023) {
        // Buat node baru
        NodeDLL_2511532023 newNode_2023 = new
NodeDLL_2511532023(newData_2023);
        // Jika dll null jadikan head
        if (head_2023 == null) {
            head_2023 = newNode_2023;
        } else {
            NodeDLL_2511532023 curr_2023 = head_2023;
            while (curr_2023.next_2023 != null) {
                curr_2023 = curr_2023.next_2023;
            }
            curr_2023.next_2023 = newNode_2023;
            newNode_2023.prev_2023 = curr_2023;
        }
        return head_2023;
    }
    // Fungsi menambahkan node di posisi tertentu
    public static NodeDLL_2511532023
insertAtPosition_2023(NodeDLL_2511532023 head_2023, int pos_2023, int
new_data_2023) {
        // Buat node baru
        NodeDLL_2511532023 new_node_2023 = new
NodeDLL_2511532023(new_data_2023);
        if (pos_2023 == 1) {
            new_node_2023.next_2023 = head_2023;
            if (head_2023 != null) {
```



```

        head_2023.prev_2023 = new_node_2023; }
        head_2023 = new_node_2023;
        return head_2023; }
NodeDLL_2511532023 curr_2023 = head_2023;
for (int i = 1; i < pos_2023 - 1 && curr_2023 != null; ++i)
{
    curr_2023 = curr_2023.next_2023; }
    if (curr_2023 == null) {
        System.out.println("Posisi tidak ada");
        return head_2023; }
        new_node_2023.prev_2023 = curr_2023;
        new_node_2023.next_2023 =
curr_2023.next_2023;
        curr_2023.next_2023 = new_node_2023;
        if (new_node_2023.next_2023 !=
null) {

            new_node_2023.next_2023.prev_2023 = new_node_2023; }
            return head_2023;
        }
        public static void printList_2023(NodeDLL_2511532023
head_2023) {
            NodeDLL_2511532023 curr_2023 = head_2023;
            while (curr_2023 != null) {

                System.out.print(curr_2023.data_2023 + " <-> ");
                curr_2023 = curr_2023.next_2023;

            }
            System.out.println();
        }
        public static void main(String[] args) {
            // Membuat dll 2 <-> 3 <-> 5
            NodeDLL_2511532023 head_2023 = new
NodeDLL_2511532023(2);
            head_2023.next_2023 = new NodeDLL_2511532023(3);
            head_2023.next_2023.prev_2023 = head_2023;
            head_2023.next_2023.next_2023 = new
NodeDLL_2511532023(5);
            head_2023.next_2023.next_2023.prev_2023 =
head_2023.next_2023;
            // Cetak DLL awal
            System.out.print("DLL Awal: ");
            printList_2023(head_2023);
            // Tambah 1 di awal
            head_2023 = insertBegin_2023(head_2023, 1);
            System.out.print("Simpul 1 ditambah di awal: ");
            printList_2023(head_2023);
            // Tambah 6 di akhir
            System.out.print("Simpul 6 ditambah di akhir: ");
            int data_2023 = 6;
            head_2023 = insertEnd_2023(head_2023, data_2023);
            printList_2023(head_2023);
            // Menambah node 4 di posisi 4
            System.out.print("Tambah node 4 di posisi 4: ");
            int data2_2023 = 4;
            int pos_2023 = 4;

```

```

        head_2023 = insertAtPosition_2023(head_2023,
pos_2023, data2_2023);
        printList_2023(head_2023);
    }
}

```

Kode Program 2.3.2

Penjelasan Kode:

- Fungsi insertBegin_2023
 - NodeDLL_2511532023 new_node_2023 = new NodeDLL_2511532023(data_2023);: Membuat instansiasi objek node baru
 - new_node_2023.next_2023 = head_2023;: Mengarahkan pointer next node baru ke head lama.
 - if (head_2023 != null) { head_2023.prev_2023 = new_node_2023; } : Jika list tidak kosong, pointer prev dari head lama diarahkan ke node baru.
 - return new_node_2023;: Mengembalikan node baru sebagai head yang baru.
- Fungsi insertEnd_2023
 - if (head_2023 == null) { head_2023 = newNode_2023; } : Jika list kosong, node baru langsung menjadi head.
 - while (curr_2023.next_2023 != null) { curr_2023 = curr_2023.next_2023; } : Melakukan iterasi hingga mencapai node terakhir.
 - curr_2023.next_2023 = newNode_2023;: Menghubungkan node terakhir ke node baru.
 - newNode_2023.prev_2023 = curr_2023;: Mengatur pointer prev node baru ke node terakhir lama.

Output:

```
Console X
<terminated> InsertDLL_2511532023 [Java Application] C:\Users\ACER\p2\pool\plugins\
DLL Awal: 2 <-> 3 <-> 5 <->
Simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

2.3.3 Kelas HapusDLL

Masuk ke bagian ketiga, kode memfokuskan tujuannya untuk melepaskan jeratan node yang tidak lagi diinginkan dari barisan, serta memastikan ikatan dari node-node yang selamat tetap utuh sempurna.

```
package pekan6_2511532023;

public class HapusDLL_2511532023 {
    // Fungsi menghapus node awal
    public static NodeDLL_2511532023 delHead_2023(NodeDLL_2511532023
head_2023) {
        if (head_2023 == null) {
            return null; }
        NodeDLL_2511532023 temp_2023 = head_2023;
        head_2023 = head_2023.next_2023;
        if (head_2023 != null) {
            head_2023.prev_2023 = null; }
        return head_2023;
    }
    // Fungsi menghapus di akhir
    public static NodeDLL_2511532023
delLast_2023(NodeDLL_2511532023 head_2023) {
        if (head_2023 == null) {
            return null; }
        if (head_2023.next_2023 == null) {
            return null; }
        NodeDLL_2511532023 curr_2023 = head_2023;
        while (curr_2023.next_2023 != null) {
            curr_2023 = curr_2023.next_2023;
        }
        // Update pointer previous node
        if (curr_2023.prev_2023 != null) {
            curr_2023.prev_2023.next_2023 = null; }
        return head_2023;
    }
    // Fungsi menghapus node posisi tertentu
    public static NodeDLL_2511532023
delPos_2023(NodeDLL_2511532023 head_2023, int pos_2023) {
        // Jika DLL kosong
        if (head_2023 == null) {
            return head_2023; }
        NodeDLL_2511532023 curr_2023 = head_2023;
        // Telusuri sampai ke node yang akan dihapus
        for (int i = 1; curr_2023 != null && i < pos_2023;
++i) {
```

```

        curr_2023 = curr_2023.next_2023; }
// Jika posisi tidak ditemukan
if (curr_2023 == null) {
    return head_2023; }
// Update pointer
if (curr_2023.prev_2023 != null) {
    curr_2023.prev_2023.next_2023 =
curr_2023.next_2023; }
if (curr_2023.next_2023 != null) {
    curr_2023.next_2023.prev_2023 =
curr_2023.prev_2023; }
// Jika yang dihapus head
if (head_2023 == curr_2023) {
    head_2023 = curr_2023.next_2023; }
return head_2023;
}
// Fungsi mencetak DLL
public static void printList_2023(NodeDLL_2511532023
head_2023) {
    NodeDLL_2511532023 curr_2023 = head_2023;
    while (curr_2023 != null) {
        System.out.print(curr_2023.data_2023 + " <->
");
        curr_2023 = curr_2023.next_2023;
    }
    System.out.println();
}
public static void main(String[] args) {
    // Buat sebuah DLL
    NodeDLL_2511532023 head_2023 = new
NodeDLL_2511532023(1);
    head_2023.next_2023 = new NodeDLL_2511532023(2);
    head_2023.next_2023.prev_2023 = head_2023;
    head_2023.next_2023.next_2023 = new
NodeDLL_2511532023(3);
    head_2023.next_2023.next_2023.prev_2023 =
head_2023.next_2023;
    head_2023.next_2023.next_2023.next_2023 = new
NodeDLL_2511532023(4);
    head_2023.next_2023.next_2023.next_2023.prev_2023 =
head_2023.next_2023.next_2023;
    head_2023.next_2023.next_2023.next_2023.next_2023 =
new NodeDLL_2511532023(5);

    head_2023.next_2023.next_2023.next_2023.next_2023.prev_2023 =
head_2023.next_2023.next_2023.next_2023;

    System.out.print("DLL Awal: ");
    printList_2023(head_2023);

    System.out.print("Setelah head dihapus: ");
    head_2023 = delHead_2023(head_2023);
    printList_2023(head_2023);

    System.out.print("Setelah node terakhir dihapus: ");
    head_2023 = delLast_2023(head_2023);
    printList_2023(head_2023);
}

```

```

        System.out.print("Menghapus node ke-2: ");
        head_2023 = delPos_2023(head_2023, 2);

        printList_2023(head_2023);
    }
}

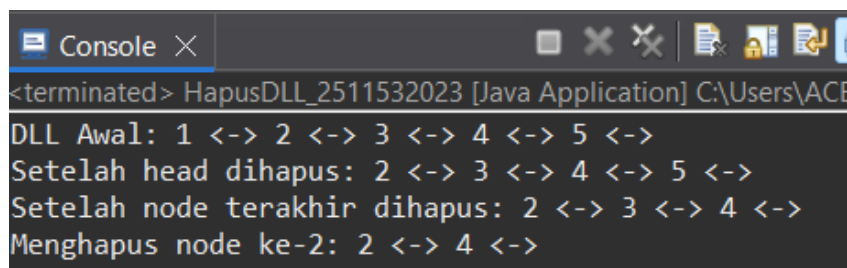
```

Kode Program 2.3.3

Penjelasan Kode:

- Fungsi delHead_2023
 - head_2023 = head_2023.next_2023;; Memindahkan referensi head ke node berikutnya.
 - if (head_2023 != null) { head_2023.prev_2023 = null; }: Menghapus referensi balik ke node yang sudah dihapus.
- Fungsi delPos_2023
 - for (int i = 1; curr_2023 != null && i < pos_2023; ++i): Iterasi mencari node pada posisi target.
 - if (curr_2023.prev_2023 != null) { curr_2023.prev_2023.next_2023 = curr_2023.next_2023; }: Memutuskan hubungan node sebelumnya dengan node saat ini dan menghubungkannya ke node setelahnya.
 - if (curr_2023.next_2023 != null) { curr_2023.next_2023.prev_2023 = curr_2023.prev_2023; }: Memutuskan hubungan node setelahnya dengan node saat ini dan menghubungkannya ke node sebelum target.

Output:



```

<terminated> HapusDLL_2511532023 [Java Application] C:\Users\ACE
DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->
Menghapus node ke-2: 2 <-> 4 <->

```

2.3.4 Kelas PenelusuranDLL

Kelas ini mengimplementasikan traversal maju dan mundur untuk membuktikan struktur dua arah.

```
package pekan6_2511532023;

public class PenelusuranDLL_2511532023 {
    // Fungsi penelusuran maju
    static void forwardTraversal_2023(NodeDLL_2511532023 head_2023) {
        // Memulai penelusuran dari head
        NodeDLL_2511532023 curr_2023 = head_2023;
        // Lanjutkan sampai akhir
        while (curr_2023 != null) {
            // Print data
            System.out.print(curr_2023.data_2023 + " <-> ");
            // Pindah ke node berikutnya
            curr_2023 = curr_2023.next_2023;
        }
        // Print spasi
        System.out.println();
    }
    // Fungsi penelusuran mundur
    static void backwardTraversal_2023(NodeDLL_2511532023 tail_2023) {
        // Mulai dari akhir
        NodeDLL_2511532023 curr_2023 = tail_2023;
        // Lanjut sampai head
        while (curr_2023 != null) {
            // Cetak data
            System.out.print(curr_2023.data_2023 + " <-> ");
            // Pindah ke node sebelumnya
            curr_2023 = curr_2023.prev_2023;
        }
        // Cetak spasi
        System.out.println();
    }
    public static void main(String[] args) {
        // Cetak DLL
        NodeDLL_2511532023 head_2023 = new NodeDLL_2511532023(1);
        NodeDLL_2511532023 second_2023 = new NodeDLL_2511532023(2);
        NodeDLL_2511532023 third_2023 = new NodeDLL_2511532023(3);

        head_2023.next_2023 = second_2023;
        second_2023.prev_2023 = head_2023;
        second_2023.next_2023 = third_2023;
        third_2023.prev_2023 = second_2023;

        System.out.println("Penelusuran maju: ");
        forwardTraversal_2023(head_2023);

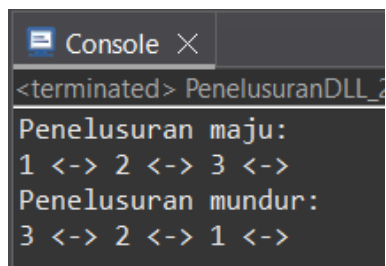
        System.out.println("Penelusuran mundur: ");
        backwardTraversal_2023(third_2023);
    }
}
```

Kode Program 2.3.4

Penjelasan Kode:

- Traversal Maju (forwardTraversal_2023): Menggunakan `curr_2023.next_2023` untuk berpindah ke node di depannya hingga mencapai null.
- Traversal Mundur (backwardTraversal_2023): Menggunakan `curr_2023.prev_2023` untuk berpindah ke node di belakangnya (memerlukan alamat tail atau node akhir).

Output:



```
<terminated> PenelusuranDLL_2
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

BAB III

PENUTUP

3.1 Kesimpulan

Praktikum ini berhasil membuktikan bahwa Double Linked List menawarkan fleksibilitas lebih baik dibandingkan Single Linked List karena kemampuan akses data dua arah. Struktur node DLL yang memiliki pointer prev dan next memungkinkan proses penghapusan dan penyisipan dilakukan secara lebih efisien tanpa harus selalu menelusuri dari head. Namun, penggunaan memori sedikit lebih besar karena adanya penyimpanan tambahan untuk dua alamat referensi pada setiap node.

3.2 Saran

Disarankan untuk melakukan pengecekan kondisi null secara ketat pada setiap operasi agar tidak terjadi error saat program dijalankan. Selain itu, penggambaran diagram alur pointer sangat dianjurkan sebelum melakukan penulisan kode untuk memastikan logika penyambungan antar node sudah tepat. Terakhir, jangan lupa untuk mengecek error pada kode program dan memastikan setiap penamaan pada variabel ditambahkan `_[NIM]` agar output yang ditampilkan sesuai dengan yang dipelajari dan penulisan kode program sesuai dengan ketentuan yang telah diberikan.

DAFTAR PUSTAKA

- [1] D. A. Fitriana, "191810101090 – DYAH AYU FITRIANA – LP7 SD KLS C," Scribd, 2020. [Daring]. Tersedia pada: <https://www.scribd.com/document/487957490/191810101090-DYAH-AYU-FITRIANA-LP7-SD-KLS-C>. [Diakses: 14-Mei-2026].
- [2] S. Dhani A, "LAPORAN RESMI DLL," Scribd, 2018. [Daring]. Tersedia pada: <https://www.scribd.com/document/394171000/LAPORAN-RESMI-DLL>. [Diakses: 14-Mei-2026].
- [3] L. P. Wibawa, "21103027 - LINTANG PRASETYA WIBAWA - MODUL 8," Studocu, 2021. [Daring]. Tersedia pada: <http://studocu.id/id/document/institut-teknologi-telkom-telkom-purwokerto/pemrograman-lanjut/21103027-lintang-prasetya-wibawa-modul-8/31986219>. [Diakses: 14-Mei-2026].
- [4] A. Fanansyah, "Praktikum Double Linked List," Scribd, 2021. [Daring]. Tersedia pada: <https://www.scribd.com/document/514199067/Praktikum-Double-Linked-List>. [Diakses: 14-Mei-2026].

TUGAS 6 PRAKTIKUM STRUKTUR DATA

1. Soal

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

2. Jawab

- Kelas Lagu_2511532023

Kelas ini mewakili satu buah node tunggal. Setiap node memiliki dua tugas utama:

- Menyimpan data: Disini atribut judul dan penyanyi berada.
- Menyimpan arah: Setiap lagu punya dua pointer, yaitu next (menunjuk ke lagu setelahnya) dan prev (menunjuk ke lagu sebelumnya).

```
package pekan6_2511532023;

import java.util.Scanner;

public class Lagu_2511532023 {
    // Atribut
    String judul_2023;
    String penyanyi_2023;
    Lagu_2511532023 next_2023;
    Lagu_2511532023 prev_2023;

    // Konstruktor
    public Lagu_2511532023(String judul_2023, String penyanyi_2023) {
        this.judul_2023 = judul_2023;
        this.penyanyi_2023 = penyanyi_2023;
        this.next_2023 = null;
        this.prev_2023 = null; }

    // Getter
    public String getJudul_2023() {
        return judul_2023; }
    public String getPenyanyi_2023() {
        return penyanyi_2023; }

    // Setter
    public void setJudul_2023(String judul_2023) {
        this.judul_2023 = judul_2023; }
    public void setPenyanyi_2023(String penyanyi_2023) {
        this.penyanyi_2023 = penyanyi_2023; }
```

```
}
```

- Kelas Musik_2511532023

Kelas Musik_2511532023 berperan dalam mengatur seluruh rangkaian nodenya. Kelas ini tidak menyimpan data lagunya secara langsung, melainkan hanya memegang ujung-ujung rantainya. Kelas ini memiliki dua pointer utama sebagai penanda batas:

- Head: selalu menunjuk ke lagu urutan pertama.
- Tail: selalu menunjuk ke lagu urutan terakhir.

➤ Logika tambahLagu

Jika tidak ada lagu (`head == null`), maka lagu baru yang masuk akan langsung menjadi head sekaligus tail. Jika ingin memasukkan lagu baru di paling belakang. Maka:

- Next dari tail yang lama disuruh memegang lagu baru.
- Prev dari lagu baru disuruh memegang balik si tail lama.
- Setelah saling berpegangan, barulah status tail dipindahkan ke lagu yang paling baru.

➤ Logika hapusLaguAwal

Saat menghapus lagu pertama, kita memutus rantai dari lagu tersebut, logikanya: posisi head digeser ke lagu urutan kedua (`head_2023 = head_2023.next_2023`). Setelah head bergeser ke lagu kedua, maka prev harus kita putus (`head_2023.prev_2023 = null`) agar ia tidak lagi menunjuk ke lagu pertama yang lama. Jika tidak diputus, lagu pertama itu akan terus menyangkut di memori.

➤ Logika tampilMaju

Kita menugaskan sebuah var pembantu seperti `curr_2023` untuk berjalan dari node pertama. `curr_2023` akan mulai dari head. Ia akan mencetak judul lagu, lalu berjalan ke gerbong selanjutnya menggunakan perintah `curr_2023 = curr_2023.next_2023`. Ia akan terus berjalan sampai `curr_2023` bernilai null, yang berarti sudah melewati tail.

➤ Logika tampilMundur

Kebalikan dari tampilMaju, var curr_2023 akan mulai dari tail. Ia akan mencetak judul lagu, lalu berjalan mundur menggunakan perintah curr_2023 = curr_2023.prev_2023, sampai ia mentok di sebelum head_2023 (bernilai null).

➤ Logika cariLagu

Logikanya sama dengan tampilMaju. Kita menyuruh curr_2023 berjalan dari head ke tail satu persatu. Bedanya ia akan membandingkan judul di setiap node untuk mencari judul yang sama dengan yang dicari oleh user. Jika ya, pencarian berhenti. Jika pencarian bernilai null dan tidak menemukan yang cocok, artinya lagunya tidak ada di playlist.

```
package pekan6_2511532023;

import java.util.Scanner;

public class Musik_2511532023 {
    // Deklarasi pointer head dan tail
    Lagu_2511532023 head_2023;
    Lagu_2511532023 tail_2023;

    // Konstruktor
    public Musik_2511532023() {
        this.head_2023 = null;
        this.tail_2023 = null;
    }

    // 1. Menambah lagu baru di akhir playlist
    public void tambahLagu_2023(String judul_2023, String
    penyanyi_2023) {
        // Membuat objek node lagu baru
        Lagu_2511532023 laguBaru_2023 = new
        Lagu_2511532023(judul_2023, penyanyi_2023);
        // Jika playlist masih kosong
        if (head_2023 == null) {
            head_2023 = laguBaru_2023;
            tail_2023 = laguBaru_2023;
        }
        // Jika sudah ada lagu, tambahkan di akhir
        else {
            tail_2023.next_2023 = laguBaru_2023;
            laguBaru_2023.prev_2023 = tail_2023;
            tail_2023 = laguBaru_2023;
        }
        System.out.println("Lagu berhasil ditambahkan! \n");
    }

    // 2. Menghapus Lagu pertama
    public void hapusLaguAwal_2023() {
        // Pengecekan agar program tidak error saat playlist kosong
        if (head_2023 == null) {
            System.out.println("Playlist kosong, tidak ada lagu
            yang bisa dihapus. \n");
        }
    }
}
```

```

        return;    }
    // Jika hanya ada satu lagu
    if (head_2023 == tail_2023) {
        head_2023 = null;
        tail_2023 = null;    }
    // Jika lagu > 1
    else {
        head_2023 = head_2023.next_2023;
        head_2023.prev_2023 = null;    }
    System.out.println("Lagu pertama berhasil dihapus!\n");
}

// 3. Menampilkan semua lagu awal ke akhir (maju)
public void tampilMaju_2023() {
    // Jika playlist kosong
    if (head_2023 == null) {
        System.out.println("Playlist masih kosong.\n");
        return;    }
    System.out.println("--- Playlist (Maju) ---");
    Lagu_2511532023 curr_2023 = head_2023;
    int no_2023 = 1;
    // Looping selaman node belum bernilai null
    while (curr_2023 != null) {
        System.out.println(no_2023 + ". " +
curr_2023.getJudul_2023() + " - " + curr_2023.getPenyanyi_2023());
        curr_2023 = curr_2023.next_2023;
        no_2023++;    }
    System.out.println(); }

// 4. Menampilkan semua lagu dari akhir ke awal (mundur)
public void tampilMundur_2023() {
    // Jika playlist kosong
    if (tail_2023 == null) {
        System.out.println("Playlist masih kosong.\n");
        return;    }
    System.out.println("--- Playlist (Mundur) ---");
    Lagu_2511532023 curr_2023 = tail_2023;
    int no_2023 = 1;
    // Looping bergerak mundur menggunakan pointer prev
    while (curr_2023 != null) {
        System.out.println(no_2023 + ". " +
curr_2023.getJudul_2023() + " - " + curr_2023.getPenyanyi_2023());
        curr_2023 = curr_2023.prev_2023;
        no_2023++;    }
    System.out.println(); }

// 5. Mencari Lagu berdasarkan judul
public void cariLagu_2023(String cariJudul_2023) {
    // Pengecekan awal jika playlist masih kosong
    if (head_2023 == null) {
        System.out.println("Playlist kosong, tidak dapat
mencari lagu.\n");
        return;    }
    Lagu_2511532023 curr_2023 = head_2023;
    boolean ditemukan_2023 = false;

    // Looping mengecek setiap lagu di playlist

```

```

        while (curr_2023 != null) {
            // Membandingkan teks judul
            if
(curr_2023.getJudul_2023().equalsIgnoreCase(cariJudul_2023)) {
                System.out.println("Lagu ditemukan: " +
curr_2023.getJudul_2023() + " oleh " + curr_2023.getPenyanyi_2023() +
"\n");
                ditemukan_2023 = true;
                break; }
            curr_2023 = curr_2023.next_2023; }
        // Jika loop selesai tapi flag masih false (lagu tidak ada)
        if (!ditemukan_2023) {
            System.out.println("Lagu dengan judul '" +
cariJudul_2023 + "' tidak ditemukan.\n");
        }
    }
    // Main class
    public static void main(String[] args) {
        Scanner sc_2023 = new Scanner(System.in);
        Musik_2511532023 playlist_2023 = new Musik_2511532023();
        int pilihan_2023;

        do {
            // Menampilkan opsi menu
            System.out.println("=== Playlist Musik NIM:
2511532023 ===");
            System.out.println("1. Tambah Lagu");
            System.out.println("2. Hapus Lagu Pertama");
            System.out.println("3. Lihat Playlist (Maju)");
            System.out.println("4. Lihat Playlist (Mundur)");
            System.out.println("5. Cari Lagu");
            System.out.println("6. Keluar\n");
            System.out.print("Pilihan: ");

            pilihan_2023 = sc_2023.nextInt();
            sc_2023.nextLine();

            switch (pilihan_2023) {
                case 1:
                    System.out.print("Judul: ");
                    String judul_2023 = sc_2023.nextLine();
                    System.out.print("Penyanyi: ");
                    String penyanyi_2023 = sc_2023.nextLine();
                    playlist_2023.tambahLagu_2023(judul_2023, penyanyi_2023);
                    break;
                case 2:
                    playlist_2023.hapusLaguAwal_2023();
                    break;
                case 3:
                    playlist_2023.tampilMaju_2023();
                    break;
                case 4:
                    playlist_2023.tampilMundur_2023();
                    break;
                case 5:
                    System.out.print("Masukkan Judul Lagu yang dicari: ");
                    String cari_2023 = sc_2023.nextLine();
                    playlist_2023.cariLagu_2023(cari_2023);

```

```

        break;
    case 6:
        System.out.println("Keluar dari program playlist...");
        break;
    default:
        System.out.println("Pilihan tidak valid, silakan coba
lagi.\n"); }
    } while (pilihan_2023 != 6);
    sc_2023.close();
}
}

```

Output:

- Program tidak error saat playlist kosong
 - Saat hapus lagu pertama

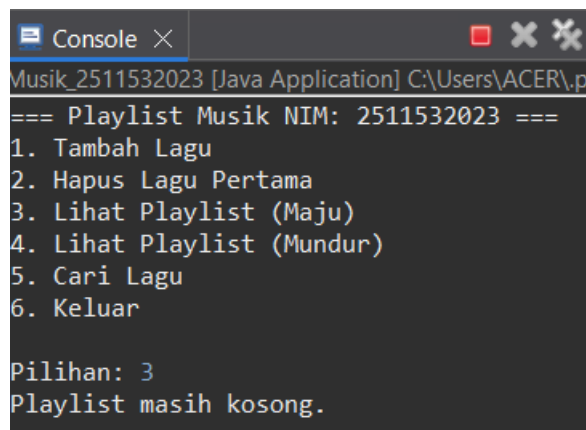
```

=== Playlist Musik NIM: 2511532023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 2
Playlist kosong, tidak ada lagu yang bisa dihapus.

```

- Saat lihat playlist



```

Console X
Musik_2511532023 [Java Application] C:\Users\ACER\p
=== Playlist Musik NIM: 2511532023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 3
Playlist masih kosong.

```

- Tambah Lagu
 - Mio Cristo Piange Diamanti

```
Console X
<terminated> Musik_2511532023 [Java Application] C:\Users\ACER\
=== Playlist Musik NIM: 2511532023 ===
<terminated> Musik_2511532023 [Java Application] C:\Users\ACER\
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 1
Judul: Rosalia - Mio Cristo Piange Diamanti
Penyanyi: Rosalia
Lagu berhasil ditambahkan!
```

◦ APT

```
Console X
<terminated> Musik_2511532023 [Java Application] C:\Users\
=== Playlist Musik NIM: 2511532023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 1
Judul: APT
Penyanyi: ROSE & Bruno Mars
Lagu berhasil ditambahkan!
```

◦ Viva La Vida

```
Console X
<terminated> Musik_2511532023 [Java Application] C:\Users\ACER\
=== Playlist Musik NIM: 2511532023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 1
Judul: Viva La Vida
Penyanyi: Coldplay
Lagu berhasil ditambahkan!
```


- Lihat Playlist (Maju)

```

Console X
<terminated> Musik_2511532023 [Java Application] C:\Users\ACER\
=== Playlist Musik NIM: 2511532023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 3|
--- Playlist (Maju) ---
1. Rosalia - Mio Cristo Piange Diamanti - Rosalia
2. APT - ROSE & Bruno Mars
3. Viva La Vida - Coldplay

```

- Lihat Playlist (Mundur)

```

Console X
<terminated> Musik_2511532023 [Java Application] C:\Users\ACER\p2\p
=== Playlist Musik NIM: 2511532023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)|
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 4
--- Playlist (Mundur) ---
1. Viva La Vida - Coldplay
2. APT - ROSE & Bruno Mars
3. Rosalia - Mio Cristo Piange Diamanti - Rosalia

```

- Cari Lagu

```

Console X
<terminated> Musik_2511532023 [Java Application] C:\Users
=== Playlist Musik NIM: 2511532023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 5
Masukkan Judul Lagu yang dicari: apt
Lagu ditemukan: APT oleh ROSE & Bruno Mars

```

- Hapus Lagu Pertama

```
Console X
<terminated> Musik_2511532023 [Java Application] C:\User

=== Playlist Musik NIM: 2511532023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 2
Lagu pertama berhasil dihapus!

=== Playlist Musik NIM: 2511532023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 3
--- Playlist (Maju) ---
1. APT - ROSE & Bruno Mars
2. Viva La Vida - Coldplay
```

- Keluar

```
=== Playlist Musik NIM: 2511532023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 6
Keluar dari program playlist...
```